

Statement

Submissions

Solution

Hints

AI Help

Java

Search an element in an array

You are given an array A of size N and an element X . Your task is to find whether the array A contains the element X or not.

Input Format

- The first line contains two space-separated integers N and X — the size of array and the element to be searched.
- The second line contains all the elements of array A

Output Format

Output "YES" if the element X is present in A , otherwise output "NO".

Constraints

- $1 \leq N, X \leq 10^5$
- $1 \leq A_i \leq 10^5$

Sample 1:

Input	Output
5 3 7 3 5 2 1	YES

```
9 // your code goes here
10 Scanner sc = new Scanner(System.in);
11 boolean flag = true;
12 int A = sc.nextInt();
13 int X = sc.nextInt();
14 int[] arr = new int[A];
15 for(int i=0;i<A;i++){
16     arr[i] = sc.nextInt();
17 }
18 for(int i=0;i<A;i++){
19     if(arr[i]==X){
20         flag = false;
21         System.out.println("YES");
22         break;
23     }
24 }
25 if(flag){
26     System.out.println("NO");
27 }
28 }
```

Test against Custom Input

Sample Input

```
5 3
7 3 5 2 1
```

Your Output

```
YES
```

[Visualize Code](#)[Run](#)[Submit](#)[Next](#)

Find maximum in an Array

Given a list of N integers, representing height of mountains. Find the height of the tallest mountain.

Input:

- First line will contain T , number of testcases. Then the testcases follow.
- The first line in each testcase contains one integer, N .
- The following line contains N space separated integers: the height of each mountains.

Output:

For each testcase, output one line with one integer: the height of the tallest mountain for that test case.

Constraints

- $1 \leq T \leq 10$
- $1 \leq N \leq 100000$
- $0 \leq \text{height of each mountain} \leq 10^9$

Sample 1:

Input	Output
1 5 4 7 6 3 1	7

```
2 import java.lang.*;
3 import java.io.*;
4
5 class Codechef
6 {
7     public static void main (String[] args) throws java.lang.Exception
8     {
9         // your code goes here
10        Scanner sc = new Scanner(System.in);
11        int T = sc.nextInt();
12        while(T-->0){
13            int N = sc.nextInt();
14            int tallest = 0;
15            int arr[] = new int[N];
16            for(int i=0;i<arr.length;i++){
17                arr[i] = sc.nextInt();
18            }
19            for(int i=0;i<arr.length;i++){
20                if(arr[i]>tallest){
21                    tallest = arr[i];
22                }
23            }
24            System.out.println(tallest);
25        }
26    }
27 }
```

Test against Custom Input

```
1
5
4 7 6 3 1
```

Visualize Code

Run

Submit

Next

Take discount or Not

There are N items in a shop. You know that the price of the i -th item is A_i . Chef wants to buy all the N items.

There is also a discount coupon that costs X rupees and reduces the cost of every item by Y rupees. If the price of an item was initially $\leq Y$, it becomes free, i.e. costs 0.

Determine whether Chef should buy the discount coupon or not. Chef will buy the discount coupon if and only if the total price he pays after buying the discount coupon is **strictly less** than the price he pays without buying the discount coupon.

Input Format

- The first line of input will contain a single integer T , denoting the number of test cases. The description of the test cases follows.
- Each test case consists of two lines of input.
 - The first line of the test case contains three space-separated integers — N , X , and Y .
 - The second line contains N space-separated integers — $A_1, A_2, \dots, A_N$.

Output Format

- For each test case, output `COUPON` if Chef should buy the discount coupon, and `NO COUPON` otherwise.

Each letter of the output may be printed in either lowercase or uppercase. For example, the strings

`coupon`, `CouPoN`, and `COUPON` will all be treated as equivalent.

```
1 class Solution {
2     public String checkCoupon(int n, int x, int y, int[] prices) {
3         // write your code here
4         int originalCost = 0;
5         int afterBuyCost = 0;
6         for(int i=0;i<n;i++){
7             originalCost += prices[i];
8             if(prices[i]>y){
9                 int value = prices[i]-y;
10                afterBuyCost += value;
11            }
12        }
13        if(afterBuyCost+x<originalCost){
14            return "COUPON";
15        }
16        return "NO COUPON";
17    }
18 }
```

Test against Custom Input

```
5
4 30 10
15 8 22 6
4 40 10
```

Visualize Code

Run

Submit

Next

Cost of Groceries

Chef visited a grocery store for fresh supplies. There are N items in the store where the i^{th} item has a freshness value A_i and cost B_i .

Chef has decided to purchase **all** the items having a freshness value **greater than equal to** X . Find the total cost of the groceries Chef buys.

Input Format

- The first line of input will contain a single integer T , denoting the number of test cases.
- Each test case consists of multiple lines of input.
 - The first line of each test case contains two space-separated integers N and X — the number of items and the minimum freshness value an item should have.
 - The second line contains N space-separated integers, the array A , denoting the freshness value of each item.
 - The third line contains N space-separated integers, the array B , denoting the cost of each item.

Output Format

For each test case, output on a new line, the total cost of the groceries Chef buys.

Constraints

- $1 \leq T \leq 100$
- $1 \leq N, X \leq 100$

```
1 class Solution {  
2     public int compute(int n, int x, int[] a, int[] b) {  
3         // write your code here  
4         int ans = 0;  
5         for(int i=0;i<n;i++){  
6             if(a[i]>=x){  
7                 ans += b[i];  
8             }  
9         }  
10        return ans;  
11    }  
12 }
```

Test against Custom Input

```
4  
2 20  
15 67  
10 90
```

Visualize Code

▶ Run

Submit

Next

[Switch to AI Tutor Mode >](#)

NEW

Check if the array is sorted

Given an array *nums* which is rotated. You have to find out if the given array is sorted and rotated.

An array is considered **sorted and rotated** if:

- There exists a non-decreasing sorted array *A*.
- After rotating *A* by some *k* positions (possibly *k* = 0), we obtain the given array *nums*.
- Rotation means some suffix of *A* is moved to the front, keeping the relative order of elements.

Duplicates are allowed in the array.

Note:

If *A* is the original sorted array and it is rotated **right** by *k* positions, the resulting array *B* satisfies:

$$B[(i + k) \bmod A.length] = A[i]$$

for every valid index *i*.

1 2 3 4 5 is a sorted array and 2 3 4 5 1 is also a sorted array but after 4 rotations.

Function Declaration

Function Name

check – This function checks whether a given array *nums* is a non-decreasing sorted array that has been rotated any number of times (including zero rotations).

```
1 public static boolean check(int[] nums) {  
2     // write your code here  
3     int count = 0;  
4     int n = nums.length;  
5     for(int i=0;i<n;i++){  
6         if(nums[i]>nums[(i+1)%n]){  
7             count++;  
8         }  
9     }  
10    return count<=1;  
11 }
```

Test against Custom Input

```
7  
6 7 1 2 3 4 5
```

Visualize Code

▶ Run

Submit

Next

[Switch to AI Tutor Mode >](#)

NEW

MIN To MAX

You are given an array A of size N .

Let M be the **minimum** value present in the array initially.

In one operation, you can choose an element A_i ($1 \leq i \leq N$) and an integer X ($1 \leq X \leq 100$), and set $A_i = X$.

Determine the **minimum** number of operations required to make M the **maximum** value in the array A .

Input Format

- The first line of input will contain a single integer T , denoting the number of test cases.
- Each test case consists of multiple lines of input.
 - The first line of each test case contains a single integer N - the size of the array.
 - The next line of each test case contains N space-separated integers $A_1, A_2, \dots, A_N$ - the elements of the array.

Output Format

For each test case, output on a new line, the **minimum** number of operations required to make M the **maximum** value in the array A .

```
1 class Solution {  
2     public int countNonMinimum(int[] nums) {  
3         // write your code here  
4         int min = Integer.MAX_VALUE;  
5         for(int i=0;i<nums.length;i++){  
6             if(nums[i]<min){  
7                 min = nums[i];  
8             }  
9         }  
10        int minCount = 0;  
11        for(int i=0;i<nums.length;i++){  
12            if(nums[i]!=min){  
13                minCount++;  
14            }  
15        }  
16        return minCount;  
17    }  
18 }
```

Test against Custom Input

```
3  
2  
1 2  
4
```

Visualize Code

▶ Run

Submit

Next

[Switch to AI Tutor Mode >](#)

NEW

Single number in multiple numbers

You are given a **non-empty** array of integers *nums*.

In this array, every number occurs **exactly twice** except for one number that occurs only once.

Your task is to find and return that unique number.

The solution must run in **O(n)** time complexity and use **O(1)** space complexity.

Function Declaration

Function Name

singleNumber – Finds the one number in the array that appears exactly once while all other numbers appear twice.

Parameters

- *nums* : A list/array of integers where every value appears exactly twice except one.

Return Value

- Returns an **integer** —
the unique number that appears only once.

Constraints:

```
1 class Solution {  
2     public int singleNumber(int[] nums) {  
3         // write your code here  
4         int xor = 0;  
5         for(int num:nums){  
6             xor ^= num;  
7         }  
8         return xor;  
9     }  
10 }
```

Test against Custom Input

```
3  
1  
10  
5
```

Visualize Code

▶ Run

Submit

Next

[Switch to AI Tutor Mode >](#)

NEW

Largest and Second Largest

You are given an array A of N integers.

Find the **maximum** sum of **two distinct** integers in the array.

Note: It is guaranteed that there exist at least two distinct integers in the array.

Input Format

- The first line of input will contain a single integer T , denoting the number of test cases.
- Each test case consists of multiple lines of input.
 - The first line of each test case contains single integer N — the size of the array.
 - The next line contains N space-separated integers, denoting the array A .

Output Format

For each test case, output on a new line, the maximum sum of two distinct integers in the array.

Constraints

- $1 \leq T \leq 1000$
- $2 \leq N \leq 10^5$
- $1 \leq A_i \leq 1000$
- The sum of N over all test cases does not exceed $2 \cdot 10^5$.

```
4 public static void main(String[] args) {
5     Scanner scanner = new Scanner(System.in);
6     int t = scanner.nextInt();
7
8     while (t-- > 0) {
9         int n = scanner.nextInt();
10        int[] a = new int[n];
11
12        for (int i = 0; i < n; i++) {
13            a[i] = scanner.nextInt();
14        }
15        // Your code goes here
16        int largest = 0;
17        int secondLargest = 0;
18        for(int i=0;i<a.length;i++){
19            if(a[i]>largest){
20                secondLargest = largest;
21                largest = a[i];
22            }
23            else if(secondLargest<a[i] && a[i]!=largest){
24                secondLargest = a[i];
25            }
26        }
27        System.out.println(largest+secondLargest);
28    }
```

Test against Custom Input

```
4
3
4 1 6
7
```

[Visualize Code](#)[Run](#)[Submit](#)[Next](#)

[Switch to AI Tutor Mode >](#)

NEW

Difficulty Rating Order

Our Chef has some students in his coding class who are practicing problems. Given the difficulty of the problems that the students have solved in order, help the Chef identify if they are solving them in non-decreasing order of difficulty. Non-decreasing means that the values in an array is either increasing or remaining the same, but not decreasing. That is, the students should not solve a problem with difficulty d_1 , and then later a problem with difficulty d_2 , where $d_1 > d_2$.

Output "Yes" if the problems are attempted in non-decreasing order of difficulty rating and "No" if not.

Input Format

- The first line of input will contain a single integer T , denoting the number of test cases. The description of the test cases follows.
- Each test case consists of 2 lines of input.
 - The first line contains a single integer N , the number of problems solved by the students
 - The second line contains N space-separate integers, the difficulty ratings of the problems attempted by the students in order.

Output Format

- For each test case, output in a new line "Yes" if the problems are attempted in non-decreasing order of difficulty rating and "No" if not. The output should be printed without the quotes.

```
5 Scanner scanner = new Scanner(System.in);
6 int t = scanner.nextInt();
7
8 while (t-- > 0) {
9     int n = scanner.nextInt();
10    int[] d = new int[n];
11
12    for (int i = 0; i < n; i++) {
13        d[i] = scanner.nextInt();
14    }
15
16    // Your code goes here
17    boolean flag = true;
18    for(int i=0;i<n-1;i++){
19        if(d[i]>d[i+1]){
20            flag = false;
21            System.out.println("No");
22            break;
23        }
24    }
25    if(flag){
26        System.out.println("Yes");
27    }
28 }
29 }
```

Test against Custom Input

```
4
3
1 2 3
3
```

Visualize Code

Run

Submit

Next